DESENVOLVIMENTO DE API BACK-END · ENGENHARIA DE SOFTWARE · 6º PERÍODO

Semana 08
Segurança: validação, senhas e OWASP

Apostila de estudo: teoria, exemplos, construção com IA e exercícios

Depois da 1ª VA, a API ganha defesas. Toda entrada é suspeita e todo id pode ser trocado por outro: um cliente curioso que muda `/service-requests/31` para `/32` não deveria ver a ordem de outra pessoa. Esta apostila apresenta o vocabulário da segurança (pilares, ameaças, autenticação e autorização, hash), as defesas na entrada (validação, senha com BCrypt, consulta parametrizada, escape de HTML), a política de mesma origem e o CORS, o OWASP API Security Top 10 e a cadeia de filtros do Spring Security 7. O login com JWT e os papéis ficam para a Semana 09.

Conteúdo

1. Objetivos da semana
2. O vocabulário da segurança
3. Autenticação e autorização
4. Superfície de ataque e ameaças
5. Codificação, criptografia e hash
6. Validação de entrada
7. Injeção de SQL
8. XSS e escape de saída
9. Mesma origem e CORS
10. OWASP API Security Top 10
11. A cadeia de filtros do Spring Security
12. Aplicação no ServiceHub
13. Construindo com IA
14. Glossário
15. Exercícios
16. Para estudar

1. Objetivos da semana

↑ conteúdo

Ao final desta semana você deve ser capaz de:

1. usar os pilares confidencialidade, integridade e disponibilidade e as quatro perguntas da modelagem de ameaças para analisar uma funcionalidade;
2. diferenciar codificação, criptografia e hash, e explicar por que a senha se guarda com um hash lento e com sal (BCrypt);
3. validar a entrada com lista branca e Bean Validation, devolvendo 400 em ProblemDetail com `errors[]`;
4. distinguir as defesas contra injeção de SQL, XSS e mass assignment, e explicar a política de mesma origem e o CORS;
5. relacionar os itens do OWASP API Security Top 10 ao código do ServiceHub e à cadeia de filtros do Spring Security 7.

2. O vocabulário da segurança

↑ conteúdo

Antes de qualquer configuração, é preciso saber o que proteger, de quem e com qual ferramenta. Sem esse vocabulário, a configuração de segurança vira receita copiada, e receita copiada protege o domínio de outra pessoa.

Os três pilares: CID

A análise de segurança parte de três propriedades, conhecidas pela sigla **CID** (em inglês, *CIA triad*). Para cada dado ou funcionalidade, pergunte qual dos três pilares está em risco.

Pilar	Significa	Exemplo geral	No ServiceHub
Confidencialidade	Só quem tem permissão lê o dado.	O extrato do banco só aparece para o titular.	O hash da senha nunca sai na resposta; ninguém vê a ordem alheia.
Integridade	O dado só muda por quem pode, do jeito previsto.	Ninguém altera o valor de uma transferência no caminho.	Só o prestador aceita a ordem; o preço é congelado na criação (RN07).
Disponibilidade	O sistema responde quando o usuário legítimo precisa.	O site de vendas não cai num dia de promoção.	Página com no máximo 100 itens; healthcheck no container.

Cada pilar tem um ataque típico: vazamento (confidencialidade), adulteração (integridade) e derrubada por excesso de requisições (disponibilidade).

Engenharia de segurança: requisito desde o início

Segurança é um atributo de qualidade, como desempenho: não se acrescenta no fim. A **engenharia de segurança** trata a segurança em todo o ciclo de vida do software. Nos *requisitos*, define-se quem pode o quê e o que é sigiloso. Na análise de *ameaças*, pergunta-se o que pode dar errado e quanto custa. No *projeto*, escolhem-se e registram-se as defesas. Nos *testes*, cada ataque imaginado vira um teste que o tenta. Corrigir uma falha de projeto depois do sistema pronto custa muito mais do que evitá-la no requisito.

NO SERVICEHUB

As regras RN do Mapa são os requisitos de segurança; os ADRs registram as decisões; os testes de integração provam cada uma. A IA ajuda a escrever código e testes, mas não sabe quem é dono de quê no seu domínio: requisitos e ameaças dependem da equipe.

3. Autenticação e autorização

↑ conteúdo

São duas perguntas diferentes, feitas nesta ordem. **Autenticação** pergunta "quem é você?" e é respondida com algo que a pessoa sabe (senha), tem (um token, um celular) ou é (digital). **Autorização** pergunta "o que você pode fazer?" e só faz sentido depois de saber quem é.

	Autenticação	Autorização
Pergunta	Quem é você?	O que você pode fazer?
Prova	Senha, token, digital	Papel, dono do recurso, regra de negócio
Falhou	401 Unauthorized	403 Forbidden
ServiceHub	Login com e-mail e senha; token nas demais chamadas	ADMIN lista usuários; só o dono edita o serviço

ANALOGIA

No aeroporto, o documento de identidade prova quem você é (autenticação). O cartão de embarque diz em qual voo você pode entrar (autorização). Ter documento válido não dá direito a embarcar em qualquer avião.

NOME CONFUSO

O status 401 se chama "Unauthorized", mas significa *não autenticado*. É um nome histórico. Erro comum em código gerado por IA: devolver 403 quando o token está ausente, ou 401 quando falta permissão.

4. Superfície de ataque e modelagem de ameaças

↑ conteúdo

A **superfície de ataque** é o conjunto de pontos por onde um atacante pode enviar dados ou acionar o sistema. Numa API, cada rota, cada parâmetro de query, cada campo do corpo, cada cabeçalho e cada porta aberta fazem parte dela. Quanto menor a superfície, menos há para defender. No ServiceHub, ela é reduzida assim: rotas públicas são só as listadas no Mapa; o Actuator, exceto o *health*, é restrito a ADMIN; o banco não tem porta exposta em produção.

A **modelagem de ameaças** é pensar no ataque antes de escrever a defesa. Um roteiro simples usa quatro perguntas:

1. **O que estamos construindo?** Por exemplo: a consulta de uma ordem de serviço por id.
2. **O que pode dar errado?** Um cliente pode ver a ordem de outro trocando o id.
3. **O que vamos fazer a respeito?** Checar no Service se quem pede participa da ordem.
4. **Fizemos um bom trabalho?** Um teste com dois usuários prova que o segundo recebe 403.

Para a pergunta 2, um roteiro clássico é o **STRIDE**, sigla em inglês de seis categorias de ameaça: falsificação de identidade (usar o token roubado de outra pessoa), adulteração (mandar `"price": 1` ao abrir uma ordem), repúdio (negar que cancelou uma ordem, sem log que prove), vazamento de informação (stack trace com nomes de tabelas na resposta), negação de serviço (pedir `size=1000000`) e elevação de privilégio (cadastrar-se com `"role": "ADMIN"`). Cada categoria se liga a um dos pilares CID ou a uma das duas perguntas da seção 3.

5. Codificação, criptografia e hash

↑ conteúdo

Três técnicas transformam dados, e só duas protegem. Confundi-las é um dos erros mais caros em segurança.

	Codificação	Criptografia	Hash
Para quê	Representar o dado em outro formato	Esconder o dado de quem não tem a chave	Gerar uma impressão digital do dado
Volta?	Sim, qualquer um, sem chave	Sim, com a chave certa	Não: é de mão única
Protege?	Não	Sim, enquanto a chave for secreta	Sim, para conferir sem guardar o original
Exemplos	Base64, URL encoding, UTF-8	AES, TLS (o S do HTTPS)	SHA-256; para senha, BCrypt
ServiceHub	Partes do JWT (Semana 09)	HTTPS em produção	`password_hash` na tabela `users`

A senha se guarda com hash. No cadastro, calcula-se o hash e grava-se só ele. No login, calcula-se o hash do que foi digitado e compara-se com o gravado. Ninguém, nem o administrador do banco, consegue ler a senha original.

ERROS COMUNS EM TUTORIAIS

"Criptografar a senha com Base64": Base64 é só codificação, e qualquer navegador a desfaz com `atob()`. Guardar a senha com criptografia reversível também é erro: se a chave vazar, vazam todas as senhas de uma vez.

Hash de senha: lento e com sal

Pense no ataque. O banco vaza. O atacante não "reverte" o hash, porque isso é impossível; ele chuta senhas comuns, calcula o hash de cada uma e compara. Com um hash rápido, como MD5 ou SHA-256, uma placa de vídeo testa um dicionário inteiro em pouco tempo. Duas defesas mudam o jogo:

- **Sal:** um valor aleatório, diferente para cada usuário, misturado à senha antes do hash. Duas pessoas com a senha `Senha123` ganham hashes diferentes, e tabelas de hashes pré-calculados deixam de servir.
- **Lentidão com custo:** o BCrypt repete o cálculo 2^{custo} vezes. Cada +1 no custo dobra o tempo. Para um login, o atraso é de uma fração de segundo; para quem chuta milhões de senhas, o ataque fica inviável.

```
$2a$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhWy
└┘└┘└──────────── sal (22) ──────────┘└──────── hash (31) ─────────┘
versão custo                    60 caracteres no total
```

O sal e o custo ficam dentro do próprio hash: não há coluna separada para eles. O `BCryptPasswordEncoder` do Spring usa custo 10 por padrão. Hash rápido é ótimo para conferir a integridade de arquivos e péssimo para senha: aqui, a lentidão é a defesa. A documentação do Spring Security também oferece Argon2, SCrypt e PBKDF2.

LIMITE DO BCRYPT

O BCrypt considera só os primeiros 72 bytes da senha. Por isso o cadastro limita a senha a 72 caracteres. Repare que o limite é em bytes: em UTF-8, uma letra acentuada ocupa mais de um byte.

6. Validação de entrada

↑ conteúdo

Três ideias distintas aparecem juntas nesta semana e costumam ser misturadas. **Validar** rejeita o que está errado (um e-mail sem arroba). **Sanitizar** neutraliza o que é perigoso (uma tag `<script>`

numa descrição). E senha não se guarda: guarda-se o **hash**.

Lista branca, não lista negra

Validar é verificar tipo, tamanho, formato e intervalo na fronteira da API, antes de qualquer regra de negócio. Há dois jeitos de escrever a regra:

	Lista negra (bloqueia o ruim)	Lista branca (aceita só o bom)
Regra	Rejeita o que parece ataque	Aceita só o que está no formato esperado
Exemplo	Proibir `"<script>"` e `"DROP"`	`role` ∈ {CLIENT, PROVIDER}; preço > 0
Falha quando	O atacante escreve de outro jeito: `<ScRiPt>`, `<img onerror=…>`	A regra está errada, e aí o erro é visível
Manutenção	Corrida sem fim contra novas variantes	Muda só quando o domínio muda

Prefira a lista branca sempre que o conjunto válido for conhecido. Um CEP brasileiro tem oito dígitos: essa é a lista branca. Um e-mail mostra que nem sempre o formato é simples, e por isso usamos a validação pronta `@Email` em vez de inventar uma expressão regular. No ServiceHub, a lista branca aparece em três lugares: Bean Validation nos DTOs, enums (categoria, papel, status) e o `SortGuard` no `?sort=`.

Bean Validation: a regra mora no DTO

A **Bean Validation** (Jakarta Validation) é uma especificação que declara regras por anotações nos campos: `@NotBlank`, `@Email`, `@Size`, `@Pattern`, `@DecimalMin`, `@Future`. No Spring Boot 4 ela vem com o `spring-boot-starter-validation`. As anotações não fazem nada sozinhas: é o `@Valid` no parâmetro do controller que dispara a validação.



Do `@Valid` ao 400: o mesmo contrato de erro da Semana 04, agora com a lista `errors`.

```
POST /api/v1/auth/register
{ "email": "maria", "role": "ADMIN", ... }

400 · application/problem+json
{
  "type": "urn:servicehub:problem:validation-error",
  "title": "Dados inválidos",
  "status": 400,
  "errors": [
    { "field": "email", "message": "E-mail inválido" },
    { "field": "role",  "message": "O papel deve ser CLIENT ou PROVIDER" }
  ]
}   (resumido: o handler também envia detail, instance e timestamp)
```

O front-end usa `field` para marcar o campo certo no formulário, e por isso as mensagens são escritas para quem usa a API, em português.

7. Injeção de SQL e consulta parametrizada

↑ conteúdo

Injeção acontece quando texto vindo do usuário é montado dentro de um comando (SQL, shell, LDAP) e passa a ser interpretado como parte do comando. O dado vira código.

Concatenado: vulnerável

```
String sql = "SELECT * FROM users"
  + " WHERE email = '" + email + "'";
// email = ' OR '1'='1
// WHERE email = '' OR '1'='1'
// devolve todos os usuários
```

Parametrizado: seguro

```
PreparedStatement ps =
conn.prepareStatement(
  "SELECT * FROM users WHERE email = ?");
ps.setString(1, email);
// comando e valor viajam separados:
// o valor nunca é interpretado como SQL
```

A defesa é separar código de dado. Escapar aspas à mão não é defesa confiável. No Spring Data, a consulta derivada (`findByEmail`), a `@Query` com `:parametro` e a `Specification` geram consultas parametrizadas. O risco volta quando alguém concatena texto numa `@Query` nativa ou em `createQuery`.

DETALHE DO LIKE

Mesmo parametrizado, no `LIKE` os caracteres `%` e `_` continuam sendo curingas. Uma busca por `%` traria tudo. Por isso, no ServiceHub, o filtro de título escapa `%` e `_` antes de montar o padrão.

8. XSS e escape de saída

↑ conteúdo

XSS (*Cross-Site Scripting*) é fazer o navegador da vítima executar um script do atacante. No tipo **armazenado**, o script fica gravado no banco e atinge todo mundo que abre a página: um prestador grava `<script>…</script>` na descrição do serviço, a API guarda o texto como veio, o front-end insere o texto como HTML e o script roda no navegador de cada cliente, com a sessão dele. No tipo **refletido**, o script vem num link e a resposta o devolve na hora.

A defesa clássica é o **escape de saída**: trocar `< > & " '` por entidades HTML (`<`, `>` …), para o navegador *mostrar* o texto em vez de executá-lo. A regra geral é escapar na saída, conforme o contexto (HTML, atributo, JavaScript, URL); bibliotecas de front-end modernas já escapam por padrão. Como a API não sabe qual front-end vai exibir o texto, o ServiceHub escapa também ao gravar o texto livre: é **defesa em profundidade**, várias camadas independentes.

Entra	É gravado
`<b>Pintura</b>`	`<b>Pintura</b>`
`Instalação elétrica`	`Instalação elétrica` (acentos intactos)

ESCAPE QUE ESTRAGA ACENTOS

Algumas bibliotecas de escape, muito citadas em tutoriais, trocam também as letras acentuadas por entidades: "é" vira `é`. O texto fica ilegível para quem não renderiza HTML, como um aplicativo de celular. Com `HtmlUtils.htmlEscape(texto, "UTF-8")`, do Spring, só os cinco caracteres de marcação são trocados.

Cada defesa cobre um ataque diferente

Defesa	Contra	No ServiceHub
Validar	Dado malformado	Bean Validation nos DTOs, 400 com `errors`
Sanitizar	XSS armazenado	`TextSanitizer` no texto livre
Parametrizar	Injeção de SQL	Spring Data e Specification; LIKE escapa `%` e `_`
Lista branca	Campo arbitrário	`SortGuard` no `?sort=`; papéis fixos no cadastro

Escapar HTML não impede injeção de SQL, e consulta parametrizada não impede XSS. Uma defesa não substitui a outra.

9. Mesma origem e CORS

↑ conteúdo

A **origem** de uma página é a combinação de **esquema + host + porta**. Pela **política de mesma origem** (*same-origin policy*), o JavaScript de uma página só lê respostas da própria origem. Isso

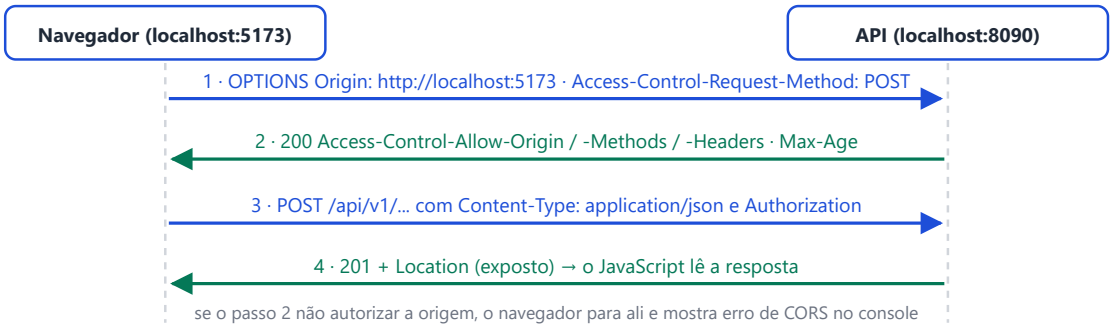
impede que um site malicioso aberto numa aba leia os dados de outro site em que você está logado.

Página em `http://localhost:5173` chama…	Mesma origem?	Por quê
`http://localhost:5173/api`	Sim	Tudo igual
`http://localhost:8090/api/v1`	Não	Porta diferente
`https://localhost:5173`	Não	Esquema diferente
`http://127.0.0.1:5173`	Não	Host diferente, mesmo sendo a mesma máquina

CORS (*Cross-Origin Resource Sharing*) é o mecanismo pelo qual o servidor diz ao navegador: "esta outra origem pode ler minhas respostas". Quem aplica a regra é o *navegador*, não o servidor. Por isso `curl` e Postman não têm CORS, e por isso CORS não é autenticação: ele não impede ninguém de chamar a API, só controla o que o JavaScript de outra origem pode ler.

O preflight

Quando a requisição não é "simples", por exemplo quando leva JSON no corpo ou o cabeçalho `Authorization`, o navegador primeiro envia sozinho uma pergunta prévia, o **preflight**: um `OPTIONS` que diz a origem, o método e os cabeçalhos pretendidos.



O fluxo do CORS com preflight entre a bancada de testes (5173) e a API (8090).

Duas consequências práticas. Primeira: o `OPTIONS` precisa estar liberado na cadeia de filtros sem exigir token, porque o navegador não envia `Authorization` no preflight. Segunda: cabeçalhos de resposta como `Location` e `X-Request-Id` só ficam visíveis ao JavaScript se o servidor os declarar como *expostos*. O Mapa exige que a API aceite a origem `http://localhost:5173`, onde roda a bancada de testes usada em sala.

ERRO COMUM

"Libere tudo com `*`." Com credenciais (`allowCredentials(true)`), o Spring recusa a combinação de origem `*`; e sem credenciais, `*` abre a API para qualquer site (OWASP API8). Use origens explícitas, vindas de variável de ambiente. E erro de CORS no console quase sempre é configuração do servidor, não bug no front-end.

10. OWASP API Security Top 10

↑ conteúdo

A **OWASP** (*Open Worldwide Application Security Project*) é uma fundação sem fins lucrativos que cataloga as falhas de segurança mais frequentes. A lista específica para APIs, na edição de 2023, é diferente do Top 10 web tradicional: aqui o primeiro lugar é autorização por objeto, e não injeção. Note que três dos cinco primeiros itens são de autorização.

Item (nome oficial)	Em uma linha
API1 · Broken Object Level Authorization	Acessar o recurso de outro só trocando o id
API2 · Broken Authentication	Login, senha ou token fracos, permitindo se passar por alguém
API3 · Broken Object Property Level Authorization	Ler ou gravar campos que não deveria (inclui mass assignment e exposição excessiva de dados)
API4 · Unrestricted Resource Consumption	Requisições sem limite de tamanho, quantidade ou frequência
API5 · Broken Function Level Authorization	Usuário comum chamando operação administrativa
API6 · Unrestricted Access to Sensitive Business Flows	Automatizar em massa um fluxo legítimo (comprar todo o estoque)
API7 · Server Side Request Forgery	Fazer o servidor buscar uma URL escolhida pelo atacante
API8 · Security Misconfiguration	Padrões abertos, CORS amplo, erros que vazam detalhes
API9 · Improper Inventory Management	Versões antigas e rotas esquecidas ainda no ar
API10 · Unsafe Consumption of APIs	Confiar cegamente no que outra API devolve

Seis itens que já atingem o ServiceHub

Item	No ServiceHub, seria	Defesa
API1 · BOLA	Ver a ordem de outra pessoa trocando o id	Posse checada no Service (RN13)
API2	Senha em texto puro, token sem validade	BCrypt; JWT com expiração (Semana 09)

Item	No ServiceHub, seria	Defesa
API3	Mandar `role` ou `provider` no corpo; ver o hash	DTOs de entrada e de saída
API4	Pedir `size=1000000`	Paginação com máximo de 100
API5	CLIENT chamando rota de ADMIN	`@PreAuthorize` no Service
API8	CORS aberto, stack trace na resposta	Origens explícitas; `include-stacktrace=never`

API1 · BOLA: o id vem do cliente, a posse vem do banco

BOLA (*Broken Object Level Authorization*) é o primeiro item porque é fácil de cometer: basta um endpoint que busca pelo id recebido e devolve o que achou. Estar autenticado não é estar autorizado. O CRUD gerado por IA quase sempre sai assim:

Vulnerável

```
@GetMapping("/{id}")
public ServiceRequest get(@PathVariable
Long id) {
  // qualquer usuário logado
  // lê qualquer ordem
  return repo.findById(id).orElseThrow();
}
```

Com checagem de posse

```
ServiceRequest r = repo.findById(id)
    .orElseThrow(...);   // 404
actorFor(r, currentUser.get());
// não participa da ordem → 403
return toResponse(r);   // DTO
```

A checagem fica no Service porque depende do banco (quem é o cliente da ordem, quem é o prestador do serviço) e precisa valer para qualquer rota que use a ordem.

API3 · Mass assignment: o corpo não escolhe dono nem papel

Mass assignment acontece quando a biblioteca de JSON preenche automaticamente todos os campos de um objeto com o que vier no corpo. Se o corpo for a entidade JPA, o cliente pode definir `id`, `provider`, `active` e `role`. O atacante tenta:

```
POST /api/v1/services
{ "title": "Pintura residencial", "price": 200.00, "category": "PINTURA",
  "provider": { "id": 1 } }

POST /api/v1/auth/register
{ ..., "role": "ADMIN" }
```

A defesa é o DTO de entrada como lista branca dos campos que o cliente pode escolher. No ServiceHub, o DTO de serviço não tem `provider`: o campo não tem onde cair, e o prestador é sempre o usuário autenticado (RN04). No cadastro, `role` só aceita CLIENT ou PROVIDER, e

qualquer outro valor dá 400 (RN03). Na saída, o DTO de usuário não tem hash (RN02). O papel só muda por `PATCH /api/v1/users/{id}/role`, uma rota de ADMIN.

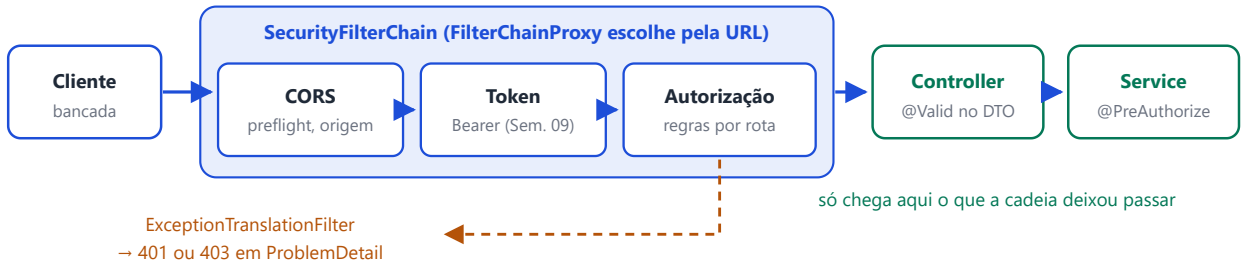
Ordenação também é entrada: o SortGuard

O Spring Data aceita qualquer caminho de atributo no `?sort=`. Sem restrição, `sort=passwordHash` ordena usuários pelo hash: a resposta não mostra o hash, mas a ordem relativa já vaza informação. É um **oráculo**, um canal que responde perguntas que não deveriam ser feitas. E `sort=provider.passwordHash` cria uma junção com a tabela de usuários; um campo inexistente vira 500. A defesa é, de novo, uma lista branca por endpoint: campo fora da lista dá 400 com os campos aceitos.

11. A cadeia de filtros do Spring Security 7

↑ conteúdo

No Spring Security, a segurança decide *antes* do seu código. Uma **cadeia de filtros** intercepta cada requisição; se ela barrar, o controller nem fica sabendo. Um **filtro** é um componente da especificação Servlet que pode inspecionar e alterar a requisição e a resposta, ou interromper o caminho.



O caminho até o controller. A ordem das regras importa: a primeira que casa com a URL vence.

Por dentro, quatro peças fazem esse trabalho: o `DelegatingFilterProxy` é a ponte entre o contêiner Servlet (Tomcat) e os beans do Spring; o `FilterChainProxy` escolhe qual `SecurityFilterChain` atende a URL; a `SecurityFilterChain` é a sua configuração (CORS, autenticação, regras por rota); e o `ExceptionTranslationFilter` transforma uma falha de segurança em 401 (não autenticado) ou 403 (sem permissão).

No Spring Security 7, a configuração é um `@Bean` do tipo `SecurityFilterChain`, montado com a API de *lambdas* (`http.csrf(csrf -> ...)`). As regras de rota usam `authorizeHttpRequests` e `requestMatchers`, e `anyRequest().authenticated()` fica por último, já que a primeira regra que casa vence. A anotação `@EnableMethodSecurity` liga o `@PreAuthorize` nos métodos dos services; sem ela, a anotação é ignorada em silêncio.

POR QUE SEM CSRF E SEM SESSÃO

CSRF (*Cross-Site Request Forgery*) é um ataque que se aproveita do cookie de sessão que o navegador envia sozinho. Uma API que não usa cookie de sessão, e recebe o token num cabeçalho em cada requisição, não tem esse cookie para ser explorado. Por isso o ServiceHub desliga o CSRF e declara a política de sessão `STATELESS`, coerente com a restrição REST "sem estado".

12. Aplicação no ServiceHub

↑ conteúdo

Os trechos abaixo são do projeto de referência (`src/main/java/com/servicehub/`). Leia-os procurando a teoria das seções anteriores.

A cadeia de filtros e o PasswordEncoder

```
// security/SecurityConfig.java (resumido)
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
  @Bean
  SecurityFilterChain securityFilterChain(HttpSecurity http, ...) throws Exception {
    http.csrf(csrf -> csrf.disable())
        .cors(cors -> { })
        .sessionManagement(s ->
            s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.OPTIONS, "/**").permitAll()
            .requestMatchers(HttpMethod.POST,
                "/api/v1/auth/register", "/api/v1/auth/login").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/v1/services", "/api/v1/services/**",
                "/api/v1/reviews/**", "/api/v1/info").permitAll()
            .requestMatchers("/actuator/health", "/actuator/health/**",
                "/actuator/info").permitAll()
            .requestMatchers("/actuator/**").hasRole("ADMIN")
            // ... documentação e /error liberados
            .anyRequest().authenticated());
        // ... resource server JWT e ProblemDetail para 401/403 (Semana 09)
    return http.build();
  }

  @Bean
  PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
}
```

CORS com origens explícitas

```
// SecurityConfig.corsConfigurationSource (resumido)
// application.properties (numa linha só):
//   servicehub.cors.allowed-origins=${CORS_ALLOWED_ORIGINS:
//       http://localhost:3000,http://localhost:5173}
config.setAllowedOrigins(origens);
config.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"));
config.setAllowedHeaders(List.of("Authorization", "Content-Type", "X-Request-Id"));
config.setExposedHeaders(List.of("Location", "X-Request-Id"));
config.setAllowCredentials(false);   // o token vai no cabeçalho, não em cookie
config.setMaxAge(3600L);             // o navegador guarda o preflight por 1 hora
```

O DTO de cadastro: a lista branca dos campos

```
// model/dto/RegisterDTO.java (resumido)
@NotBlank(message = "O e-mail é obrigatório")
@Email(message = "E-mail inválido")
@Size(max = 255, message = "O e-mail deve ter no máximo 255 caracteres")
private String email;

@NotBlank(message = "A senha é obrigatória")
@Size(min = 8, max = 72, message = "A senha deve ter entre 8 e 72 caracteres")
@Pattern(regexp = "^(?=.*[A-Z])(?=.*\\d).+$",
         message = "A senha deve conter ao menos uma letra maiúscula e um número")
private String password;

@NotBlank(message = "O papel é obrigatório")
@Pattern(regexp = "CLIENT|PROVIDER", message = "O papel deve ser CLIENT ou PROVIDER")
private String role;
```

Cadastro e login: só o hash vai ao banco

```
// service/UserService.register (resumido)
String email = normalizeEmail(dto.getEmail());          // trim + minúsculas (RN01)
if (userRepository.existsByEmail(email)) throw new EmailAlreadyExistsException(email);
user.setPasswordHash(passwordEncoder.encode(dto.getPassword()));   // RN02
user.setRole(Role.valueOf(dto.getRole()));
return toResponseDTO(userRepository.saveAndFlush(user));          // DTO sem hash

// service/AuthService.login (resumido)
boolean matches = passwordEncoder.matches(dto.password(),
        found.map(User::getPasswordHash).orElse(DUMMY_HASH));
if (found.isEmpty() || !matches || !found.get().isActive())
    throw new BadCredentialsException("E-mail ou senha inválidos.");
```

Dois detalhes de segurança no login. O `DUMMY_HASH` é um hash BCrypt fixo: quando o e-mail não existe, o BCrypt roda mesmo assim, e o tempo de resposta não revela quais contas existem. E a mensagem é a mesma para e-mail inexistente, senha errada ou conta inativa. Nunca se compara senha com `equals`: usa-se `matches`.

Sanitização do texto livre

```
// service/TextSanitizer.java
static String clean(String value) {
    if (value == null) return null;
    String trimmed = value.trim();
    return trimmed.isEmpty() ? null
        : HtmlUtils.htmlEscape(HtmlUtils.htmlUnescape(trimmed), "UTF-8");
}
```

O `TextSanitizer` é aplicado no Service a `description`, `bio`, `notes` e `comment`. O `htmlUnescape` antes do escape torna a operação idempotente: reenviar num `PUT` o valor recebido num `GET` não vira `<`.

Lista branca no `?sort=`

```
// controller/SortGuard.java (resumido) e uso em ServiceOfferingController
static final Set<String> SORTABLE = Set.of("id", "title", "price", "category", "createdAt");

static Pageable requireAllowed(Pageable pageable, Set<String> allowed) {
    for (Sort.Order order : pageable.getSort())
        if (!allowed.contains(order.getProperty()))
            throw new InvalidRequestException("Não é possível ordenar por '"
                + order.getProperty() + "'. Campos aceitos: " + new TreeSet<>(allowed));
    return pageable;
}
```

Checagem de posse (BOLA)

```
// service/ServiceRequestService.actorFor (resumido)
if (me.isAdmin()) return Actor.ADMIN;
if (request.getService().getProvider().getId().equals(me.id())) return Actor.PROVIDER;
if (request.getClient().getId().equals(me.id())) return Actor.CLIENT;
throw new AccessDeniedException("Você não participa desta ordem de serviço.");   // 403, RN13
```

O mesmo padrão aparece para o perfil (o próprio usuário ou ADMIN) e para o serviço (o dono ou ADMIN, RN05). Os testes de BOLA com dois usuários entram na Semana 09, quando houver token.

13. Construindo com IA

↑ conteúdo

A IA gera segurança *plausível*, não a segurança do seu domínio: ela não sabe quem é dono de quê, quais rotas são públicas ou que papel não pode ser escolhido no cadastro. Por isso, nesta semana, a especificação vem inteira do Mapa, e a prova é um teste que tenta o ataque, não só o caminho feliz.

Movimento	Nesta semana
Especificar	Liste antes: rotas públicas do Mapa, RN01 a RN05, RN13 e a origem `http://localhost:5173`.
Gerar	Peça um plano primeiro; depois o cadastro, em passos pequenos, com as versões explícitas (Spring Security 7, Boot 4.1).
Revisar	Algum DTO devolve o hash? `role` ou `provider` vêm do corpo? Há `*` no CORS? O `sort` aceita tudo?
Provar	Testes que atacam: cadastro com `role` ADMIN dá 400; `?sort=passwordHash` dá 400; HTML volta escapado.

Prompt fraco e prompt bom

Fraco

```
adiciona segurança e login na minha API
spring
```

Sem versão, a IA tende a usar APIs removidas. Sem regras, ela escolhe as rotas públicas, aceita qualquer papel e devolve a entidade com o hash.

Bom

Delimita o escopo (só o cadastro), fixa as versões, cita as regras RN01–RN03, proíbe explicitamente devolver o hash e aceitar ADMIN, pede plano antes do código e define os testes que provam cada regra. Veja o prompt completo abaixo.

O prompt desta semana

Ele continua a construção da semana 05: a tabela `users` já existe por migração, os DTOs e o `GlobalExceptionHandler` com ProblemDetail já funcionam.

```
Contexto: API ServiceHub em Java 21 e Spring Boot 4.1 (starters por
recurso), PostgreSQL 16 com Flyway e ddl-auto=validate. A tabela users
já existe (colunas full_name, email, password_hash, role, active...).
Já temos DTOs em records/classes e um GlobalExceptionHandler
(@RestControllerAdvice) que devolve ProblemDetail, com errors[] de
{field, message} no 400. O contrato é o Mapa de Construção:
RN01 (e-mail único, sem diferenciar maiúsculas), RN02 (senha só como
```

```
hash BCrypt, nunca em resposta), RN03 (no cadastro, papel só CLIENT
ou PROVIDER).

Objetivo: implementar o cadastro POST /api/v1/auth/register.
1. Dependências: spring-boot-starter-security e
   spring-boot-starter-validation (se ainda não houver).
2. RegisterDTO com Bean Validation e mensagens em português:
   fullName 2-100; email @Email até 255; password 8-72 com ao menos
   uma maiúscula e um número; role @Pattern "CLIENT|PROVIDER".
3. UserService.register (@Transactional): normaliza o e-mail
   (trim + minúsculas); se já existir, lança EmailAlreadyExists
   (409); grava passwordEncoder.encode(senha) em passwordHash.
4. SecurityConfig com @Bean PasswordEncoder (BCryptPasswordEncoder)
   e @Bean SecurityFilterChain: csrf desligado, sessão STATELESS,
   OPTIONS /** e POST /api/v1/auth/register liberados, GET de
   /api/v1/services e /api/v1/services/** liberados,
   anyRequest().authenticated() por último.
5. TextSanitizer (HtmlUtils.htmlUnescape e depois
   HtmlUtils.htmlEscape(texto, "UTF-8")) aplicado no Service ao texto
   livre (bio do usuário, description do serviço).
6. Resposta 201 com Location e UserResponseDTO.

Restrições:
- NUNCA devolver password nem passwordHash em resposta nenhuma; não
  usar a entidade User como corpo de entrada nem de saída.
- NÃO aceitar role ADMIN no cadastro: deve dar 400 com errors[]
  apontando o campo role (não 403, não ignorar em silêncio).
- Não use WebSecurityConfigurerAdapter, antMatchers nem
  authorizeRequests (removidos); nem StringEscapeUtils; nem MD5 ou
  SHA para senha; nem pacotes javax.*.
- Sem login nem JWT ainda (próxima etapa).
- Não altere migrações já aplicadas.

Antes de escrever código, mostre um plano: arquivos a criar ou
alterar e a lista de testes. Espere o meu ok.

Critério de aceite (cada regra com um teste):
- @WebMvcTest: role "ADMIN" → 400 com errors contendo field "role";
  email "maria" → 400 com field "email".
- Teste do UserService: o hash gravado começa com "$2" e é diferente
  da senha; "MARIA@X.COM" quando "maria@x.com" existe → 409.
- Teste do TextSanitizer: "<script>" sai "&lt;script&gt;" e
  "Instalação elétrica" sai igual.
- ./mvnw verify passa.
```

Para fechar a entrega, peça em seguida, no mesmo estilo, o `SortGuard` (lista branca por controller, 400 com os campos aceitos) e o `CorsConfigurationSource` com origens vindas de `CORS_ALLOWED_ORIGINS`, incluindo `http://localhost:5173`, expondo `Location` e `X-Request-Id`.

O que a IA costuma errar nesta etapa

A IA sugere	No ServiceHub
`extends WebSecurityConfigurerAdapter`	Removido; `SecurityFilterChain` como `@Bean`.
`antMatchers`, `authorizeRequests`	`requestMatchers`, `authorizeHttpRequests`.
`@PreAuthorize` sem `@EnableMethodSecurity`	A anotação é ignorada, sem erro nenhum.
Biblioteca de escape que troca acentos	`HtmlUtils.htmlEscape(texto, "UTF-8")`.
`allowedOrigins("*")` com credenciais	Origens explícitas; o Spring recusa a combinação.
Devolver a entidade `User`	`UserResponseDTO`, sem `passwordHash`.
`user.getPassword()`	O campo é `passwordHash`.
`@WebMvcTest` que falha com 401/403	Os filtros de segurança entram no teste fatiado; para testar só a validação, use `@AutoConfigureMockMvc(addFilters = false)`.

As duas primeiras linhas vêm de tutoriais de versões antigas do Spring Security; o assistente as repete quando o contexto não diz a versão.

Checklist de revisão

☐ Nenhum DTO de resposta tem `password` ou `passwordHash`?

☐ `role` fora de CLIENT e PROVIDER dá 400, e não 201 nem 403?

☐ O `PasswordEncoder` é BCrypt e a comparação usa `matches`?

☐ Há algum SQL montado com concatenação de texto?

☐ O escape preserva acentos e não acumula num `PUT`?

☐ CORS sem `*`, com `http://localhost:5173` e `OPTIONS` liberado?

☐ A ordem das regras da cadeia termina em `anyRequest().authenticated()`?

Prova

Os testes do critério de aceite passam em `./mvnw verify`. Com a API no ar, três chamadas confirmam o comportamento (porta local 8080; no Compose, 8090):

```
curl -i -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"fullName":"Esperto","email":"e@x.com","password":"Senha123","role":"ADMIN"}'
# esperado: 400, errors com field "role"

curl -i -X POST http://localhost:8080/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"fullName":"Maria Silva","email":"maria@x.com","password":"Senha123","role":"CLIENT"}'
# esperado: 201, Location, corpo sem password e sem passwordHash

curl -i -X OPTIONS http://localhost:8080/api/v1/auth/register \
  -H "Origin: http://localhost:5173" -H "Access-Control-Request-Method: POST"
# esperado: Access-Control-Allow-Origin: http://localhost:5173
```

No banco, `SELECT password_hash FROM users WHERE email = 'maria@x.com'` deve devolver um valor que começa com `$2a$10$`.

Entrega da semana

- `SecurityFilterChain` com as rotas públicas do Mapa e CORS para `http://localhost:5173`.
- `POST /api/v1/auth/register` com BCrypt e `role` só CLIENT ou PROVIDER; nenhuma resposta traz o hash.
- Bean Validation em todos os DTOs de entrada, com 400 em ProblemDetail e `errors`.
- Sanitização do texto livre e lista branca no `?sort=`, cada uma com um teste que tenta o ataque.

14. Glossário

↑ conteúdo

CID	Confidencialidade, integridade e disponibilidade: os pilares da segurança.
Autenticação	Verificar quem é o usuário. Falha: 401.
Autorização	Verificar o que o usuário pode fazer. Falha: 403.
Superfície de ataque	Conjunto de pontos por onde um atacante pode interagir com o sistema.
Modelagem de ameaças	Analisar o que pode dar errado antes de projetar as defesas.
Hash	Função de mão única que gera uma impressão digital do dado.
Sal	Valor aleatório por usuário misturado à senha antes do hash.
BCrypt	Algoritmo de hash de senha lento, com sal e fator de custo.
Lista branca	Validação que aceita só o que está no formato esperado.
Injeção	Ataque em que dado do usuário é interpretado como comando.
XSS	Execução de script do atacante no navegador da vítima.
Escape	Troca de caracteres especiais por entidades para que sejam exibidos, não executados.
Origem	Esquema + host + porta de uma página ou API.
CORS	Mecanismo pelo qual o servidor autoriza outra origem a ler suas respostas no navegador.
Preflight	Requisição OPTIONS que o navegador envia antes da requisição real.
BOLA	Falha de autorização por objeto: acessar o recurso de outro trocando o id.

15. Exercícios

↑ conteúdo

1. Diga qual pilar (confidencialidade, integridade ou disponibilidade) é atingido em cada caso: (a) a resposta de `GET /api/v1/users/7` traz o hash da senha; (b) um cliente consegue mudar o `totalPrice` de uma ordem já criada; (c) um script pede milhares de vezes `?size=100` por segundo.
2. Dê o status (401 ou 403) e justifique: (a) requisição sem cabeçalho `Authorization` para `GET /api/v1/service-requests`; (b) um CLIENT autenticado chama `GET /api/v1/users`; (c) token com assinatura inválida.
3. Um colega afirma: "nossa senha está segura, ela é criptografada com Base64 antes de ir para o banco". Corrija a afirmação usando os conceitos de codificação, criptografia e hash. Em seguida, explique o que significam as partes `$2a$` e `$10$` de um hash BCrypt e o que muda se o custo for 12.

4. Escreva regras de lista branca, com anotações de Bean Validation, para: (a) o preço de um serviço; (b) a nota de uma avaliação, de 1 a 5; (c) o papel no cadastro. Por que uma lista negra que bloqueia `<script>` não seria suficiente?

5. Analise os dois trechos abaixo:

```
// trecho A
String sql = "SELECT * FROM services WHERE title LIKE '%" + q + "%'";
List<Map<String, Object>> rows = jdbcTemplate.queryForList(sql);

// trecho B
em.createQuery("FROM User u WHERE u.email = :email", User.class)
  .setParameter("email", email);
```

   (a) Qual deles é vulnerável a injeção? Mostre um valor de `q` que explora a falha. (b) Reescreva o trecho vulnerável com Spring Data, sem SQL concatenado. (c) Que cuidado extra a busca com `LIKE` exige mesmo parametrizada?

6. Um prestador grava a descrição `Ótimo <b>preço</b>`. Mostre o que o `TextSanitizer` grava e explique: (a) por que usar `HtmlUtils` com `"UTF-8"`; (b) para que serve o `htmlUnescape` antes do escape; (c) se o escape protege contra injeção de SQL.

7. A bancada de testes, em `http://localhost:5173`, faz `POST` com JSON e `Authorization` para a API em `http://localhost:8090`. (a) São a mesma origem? (b) Que requisição o navegador envia antes do POST, e por quê? (c) O que acontece se a cadeia de filtros exigir token no `OPTIONS`? (d) Por que o Postman funciona mesmo sem CORS configurado?

8. Reescreva o endpoint vulnerável da seção 10 para que um cliente que não participa da ordem receba 403. Diga em qual camada fica a checagem e por quê.

9. Um atacante envia `POST /api/v1/services` com `"provider": {"id": 1}` e `POST /api/v1/auth/register` com `"role": "ADMIN"`. Descreva o que o ServiceHub faz em cada caso, citando a regra RN e o item do OWASP.

10. Explique por que `GET /api/v1/users?sort=passwordHash,desc` é perigoso mesmo sem mostrar o hash, e o que o `SortGuard` devolve.

11. Aplique as quatro perguntas da modelagem de ameaças à funcionalidade "o cliente avalia a ordem concluída" (`POST /api/v1/service-requests/{id}/reviews`). Cite pelo menos duas ameaças, com a defesa e o teste de cada uma.

12. Um assistente gerou o código abaixo. Liste os problemas e escreva o trecho de prompt que teria evitado cada um.

```
@Configuration
public class Seg extends WebSecurityConfigurerAdapter {
  protected void configure(HttpSecurity http) throws Exception {
    http.authorizeRequests().antMatchers("/api/**").permitAll();
    http.cors().configurationSource(r -> {
      CorsConfiguration c = new CorsConfiguration();
      c.addAllowedOrigin("*"); c.setAllowCredentials(true); return c; });
  }
}
```

16. Para estudar

↑ conteúdo

Leituras disponíveis na Minha Biblioteca, pelo acesso institucional:

- PRESSMAN, R. S.; MAXIM, B. R. *Engenharia de software*. 9. ed. Grupo A, 2021. Capítulo 18 (engenharia de segurança).
- RODRIGUES, T. N. et al. *Integração de aplicações*. Grupo A, 2020. Capítulo "Segurança em webservices".
- MARTIN, R. C. *Arquitetura limpa*. Alta Books, 2019. Capítulo 20 (regras de negócio): as regras de posse e de papel são regras de negócio e moram no Service, não no controller.

Referências abertas:

- OWASP API Security Top 10 (2023): `owasp.org/API-Security`. Leia ao menos API1, API3 e API8.
- OWASP Cheat Sheet Series: *Password Storage*, *Input Validation* e *Cross Site Scripting Prevention* (`cheatsheetseries.owasp.org`).
- Documentação oficial do Spring Security: arquitetura de filtros, `PasswordEncoder` e CORS (`docs.spring.io/spring-security`).
- Documentação de CORS da MDN Web Docs (`developer.mozilla.org`, "Cross-Origin Resource Sharing").
- Mapa de Construção do ServiceHub, em `docs/alunos`: regras RN01–RN13 e seção de CORS.